

PROBLEM

Predicting Customer Churn at Infrastructure Scale

- **Business problem:**

telecom churn costs 5-25x more to replace than retain; predict the 14% of customers who will leave from usage behavior.

- **Dataset:**

Churn_Dataset.csv: 5,000 customers, 16 numeric features, 341 KB. Model: MLP 128-64-32 (JAX/Flax + PyTorch), 91-94% test accuracy vs 86% majority baseline.

- **Resource challenge:**

the dataset is tiny but the pipeline must be production-shaped: every hardware tier is oversized, so FIXED costs (compile, kernel dispatch, staging) dominate.

- **Question:**

which rung of the hardware ladder (1 CPU core → many cores → GPU → TPU slice) does this workload actually need, and what is the real bottleneck?

- **Scope delivered:**

15 measured configs across 4 architectures (Apple M4, arm64 Grace, x86 Xeon, TPU v5e) and 3 sites (local, Stanford hpcc cluster, Google Cloud).

Immutable Containers, Declarative Orchestration, XLA

- **Environment (Docker/Apptainer):**

pinned images (CPU, CUDA 12.4, arm64/cu128); zero installs on nodes. SLURM runs the same pinned image as a read-only 3.2 GB .sif; K8s jobs bootstrap pinned wheels in ephemeral containers.

- **Orchestration (K8s + SLURM):**

on-prem K8s Jobs with exact requests/limits (32c Grace slice, 1×NVIDIA GPU) · GKE Autopilot TPU v5e 2x2 provisioned on demand from a nodeSelector · single-node SLURM (slurmctld+slurmd+munge) installed and configured on hpcc-cluster-49.

- **Storage tier:**

NFS / ConfigMap / GCS canonical copy → staged once to node-local scratch → RAM → device memory. The training loop never touches shared storage; the IO bus sees the CSV once per job.

- **Compilation (JAX/XLA):**

entire train step (fwd + bwd + Adam) jit-fused into one XLA computation on CPU and TPU; PyTorch eager (+ optional Inductor) on the GPU path; bf16 as a measured ablation.

MEASUREMENTS

Telemetry: Honest Step Times + Node Metrics

- **Scaling matrix:**

core sweeps (1/8/10/32) on three CPU architectures, batch sweeps (128→16,384) on GPU and TPU, bf16 ablation: 15 measured configs, 400 epochs each, one JSON record per run.

- **Step timing:**

per-step wall clock with forced device sync (`jax.block_until_ready` / `torch.mps.synchronize` / TPU); first traced+compiled step timed separately = compile overhead.

- **Node telemetry:**

background psutil sampler (whole-node CPU %, RSS); GPU memory via driver; nvidia-smi 1 Hz hook in the cluster GPU job; environment bootstrap timed (25-36 s ephemeral, 3.2 GB .sif once).

- **IO metrics:**

staged + device-resident by design → zero steady-state input stalls, verified on every backend (one 341 KB read per job).

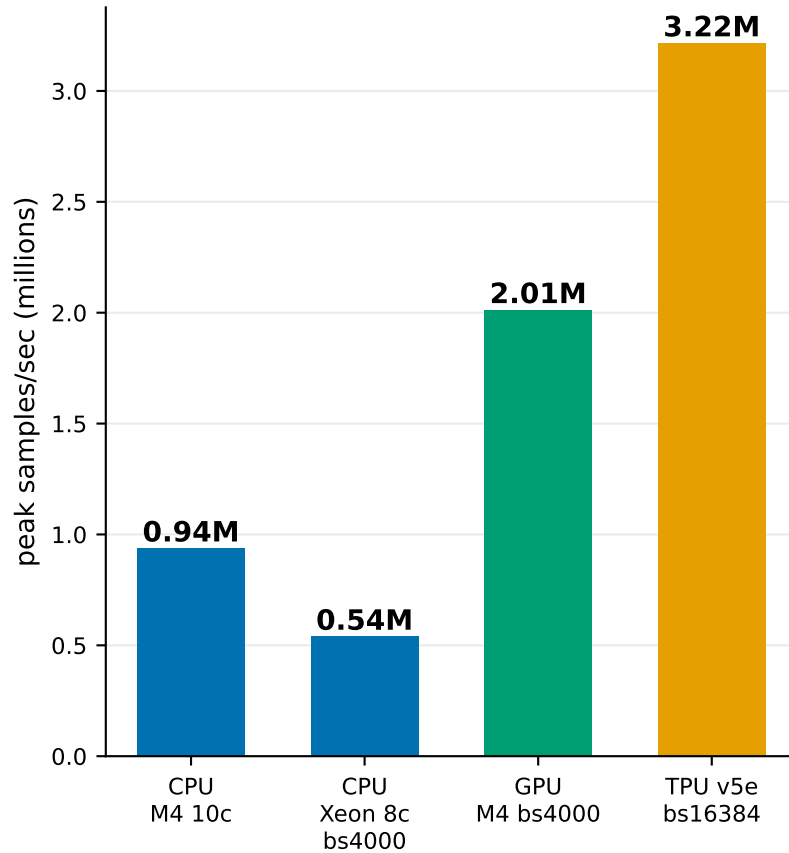
- **Automation:**

`benchmarks/run_matrix.sh`, watch-and-heal job loops, `fetch_cluster_results.py`: every number in this deck regenerates from committed JSON.

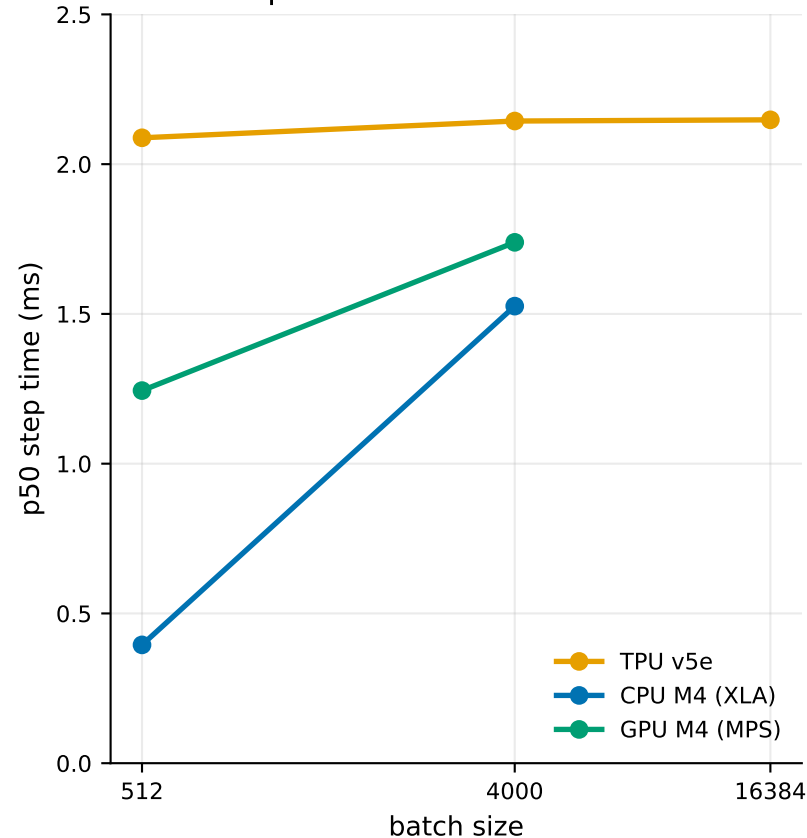
RESULTS - CPU vs GPU vs TPU

Cores Buy Nothing; Batch Size Buys Everything

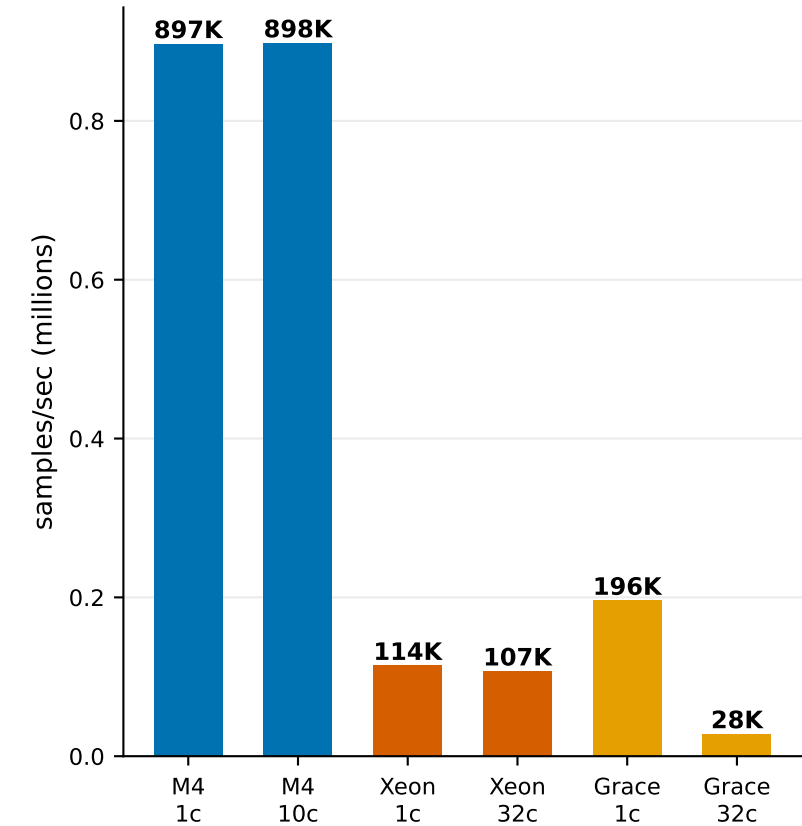
Peak throughput per backend



Step time vs batch: TPU is flat



Adding cores: flat or negative



Speedups (bs512 → best): GPU 5.15x via batch alone · TPU 17.0x via batch alone (constant 2.1 ms step) · 10x/32x cores: 1.00x / 0.94x / 0.14x.
Xeon 32-core run: 99.7% node utilization, less throughput than 1 core. Utilization is not throughput.

CONCLUSION

The Bottleneck Is Dispatch Latency, Not Hardware

■ Bottleneck identified:

fixed per-step host-side launch cost, confirmed on four architectures: core-count indifference (M4), negative core scaling (Grace 0.14x, Xeon at 100% util), and a TPU step time constant across a 32x batch range.

■ What worked:

batch scaling (GPU 5.15x, TPU 17.0x), XLA train-step fusion, device-resident data (zero IO stalls). What didn't: more cores ($\leq 1.00x$ everywhere), bf16 on CPU (0.75x), accelerators at default batch (0.2-0.4x).

■ Cost trade-off:

TPU v5e wins throughput 3.4x over the best CPU but costs $\sim 10x$ a CPU node-hour and 7 min of provisioning; below $\sim 1M$ rows a single CPU node with XLA is cost-optimal for one job.

■ Scaling recommendation:

grow batch (with LR warmup) before growing hardware; fuse epochs with `jax.lax.scan`; spend accelerator manifests on parallel hyperparameter sweeps, where the TPU's 3.22M samples/s actually converts to wall-clock wins.

■ Lesson:

measure fixed costs before buying parallelism: the scaling matrix falsified three 'obvious' upgrades (more cores, GPU at default batch, bf16-everywhere) with under a minute of total compute.